Chapter 8

The computer's CPU and RAM

This chapter gives a brief description of modern computer hardware, in particular the central processing unit (CPU) and its interaction with computer memory (RAM and caches), and some of the consequences for both sequential and parallel program performance.

We start by describing a simple single-core CPU that fetches instructions one by one from RAM and executes them, we give an example of how a Java program is executed on such a CPU, we describe the memory cache hierarchy that connects the CPU to RAM, we describe a more realistic modern CPU core's structure and its internal parallelism, we describe its instructions for memory access, and finally we describe how a CPU with many such modern cores and multiple caches ensure that these caches agree with each other.

Sections 8.5 through 8.7 are relevant only for parallel programming performance and can be skipped if you are mostly interested in single-threaded programs.

8.1 A naive computer model

We start by presenting a rather naive computer model, which is nevertheless a quite accurate description of many computers made between 1945 and 1990. Our naive computer consists of a central processing unit (CPU) directly connected to random access memory (RAM), as illustrated by figure 8.1.



Figure 8.1: An Arm1 processor from 1985 where the CPU is directly connected to RAM. The CPU has only one core and no caches.

The RAM holds data and program code, and can be thought of as an array of numbered memory locations, denoted `mem[0]`, `mem[1]`, and so on. Each memory location holds a number, which can represent either a data item, a program instruction in numeric form, or the numeric

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

address of some other location in memory, usually called a pointer. The CPU has several registers that can hold a number or a memory address (pointer). It also has a special program counter register (PC) which holds the address of the next program instruction to execute. A machine code program consists of a sequence of instructions encoded as numbers.

Consider a small program fragment in machine code for such a naive CPU. It has three instructions, stored at RAM addresses 0x000, 0x004 and 0x008 because the numeric encoding of each instruction takes up 4 bytes, and memory is byte addressed:

```
0x000: add  w20, w20, 1
0x004: cmp  w20, w10
0x008: b.lt 0x000
0x00C: ...
```

The program addresses are written in hexadecimal using the digits 0–9 and A–F; for instance, 0x008 means 8 and 0x00C means 12 in decimal.

The program repeatedly adds 1 to the `w20` register so long as its value is less than the `w10` register.

The "`add w20, w20, 1`" instruction computes the sum of register `w20` and 1 (on the right) and then stores the result in register `w20` (on the left).

The "`cmp w20, w10`" instruction compares registers `w20` and `w10`, recording the outcome in a special flags register.

The "`b.lt 0x000`" instruction is a conditional branch, which jumps to the instruction at 0x000 if `w20` was less than (`lt`) register `w10`, and otherwise continues with the next instruction at 0x00C. Such a jump in the program execution is achieved simply by assigning 0x000 to PC if the less-than condition is true.

In general, this section's naive CPU executes the program instructions one at a time, by forever executing these two steps:

- Fetch instruction: `instr = mem[PC]`, then add 4 to PC.
- Decode and execute the instruction `instr`; decide whether it is an `add`, `cmp`, `b.lt` or some other instruction, and update registers and memory accordingly.

An example execution of the code above is shown in figure 8.2.

PC	w10	w20	Instruction to execute
0x000	2	0	`add w20, w20, 1`
0x004	2	1	`cmp w20, w10`
0x008	2	1	`b.lt 0x000`
0x000	2	1	`add w20, w20, 1`
0x004	2	1	`cmp w20, w10`
0x008	2	2	`b.lt 0x000`
0x00C	2	2	...

Figure 8.2: Execution of the machine code program. The first three columns show the register values before the instruction is executed, so initially register `PC` contains 0x000, register `w10` contains 2, and register `w20` contains 0.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

8.2 How a Java program is compiled and executed

This example Java for-loop assumes that `k` is an `int` variable and `arr` is an integer array. It repeatedly sets `k` to the contents of the array element `arr[k]`:

```
for (int j=0; j<arr.length; j++)
  k = arr[k];
```

8.2.1 Java bytecode

When we compile the above loop with a Java compiler (`javac`), we get a .class file containing among other things this Java bytecode, corresponding to the for-loop:

```
 4: iload_2
 5: aload_0
 6: arraylength
 7: if_icmpge 20
10: aload_0
11: iload_1
12: iaload
13: istore_1
14: iinc 2, 1
17: goto 4
```

The code uses three local variables, where number 0 is `arr` from the Java program, number 1 is `k` and number 2 is `j`. The variable numbers are used in the instruction names such as `iload_2`.

The instruction at address 4 loads integer `j` onto the evaluation stack. The instructions at address 5 and 6 load `arr` onto the stack, computes its length `arr.length`, and leaves that on the stack top. Instruction 7 compares `j` and `arr.length` and if the result is greater-or-equal (`ge`) it jumps to address 20, thereby terminating the loop. Instruction 10 loads `arr` onto the evaluation stack, instruction 11 loads `k` and instruction 12 performs an array element load, replacing `arr` and `k` with `arr[k]`, and implicitly checks that `k` is within the array bounds. Instruction 13 stores the loaded value into `k`. Instruction 14 increments `j` by 1 and instruction 17 jumps back to the start of the for-loop at 4.

8.2.2 Just-in-time compilation to machine code

When we run the bytecode shown in section 8.2.1 using the Java runtime system (`java`), the Java just-in-time compiler will generate real machine code from the bytecode. On a CPU with the Arm64 instruction set, as in a modern Macbook or Raspberry Pi, the resulting machine code might look like this:

```
0x38c: cmp  w19, w13
0x390: b.cs 0x4dc
0x394: add  x11, x21, w19, sxtw #2
0x398: add  w20, w20, #0x1
0x39c: ldr  w19, [x11, #16]
0x3a0: cmp  w20, w10
0x3a4: b.lt 0x38c
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

The Arm64 machine code uses registers `w0`–`w30` and `x0`–`x30` instead of the JVM bytecode's evaluation stack. In our example, the relation between the machine code registers and the Java program is shown in the table below. (It is somewhat wasteful that the same `arr.length` value is held in two distinct registers; perhaps a future version of the Java just-in-time compiler will avoid that).

Register	Bytecode var	Value in Java program
`w10`		`arr.length`
`x11`		computed address of `arr[k]`
`w13`		`arr.length`
`w19`	1	`k`
`w20`	2	`j`
`x21`	0	base address of array `arr`

The machine code instructions have the following meaning:

Instruction	Meaning
`0x38c cmp w19, w13`	compare `k` and array length
`0x390 b.cs 0x4dc`	if `k` out of bounds, jump to throw exception
`0x394 add x11, x21, w19, sxtw #2`	compute `x11 = x21 + 4 * k`
`0x398 add w20, w20, #0x1`	compute `w20 = w20 + 1`
`0x39c ldr w19, [x11, #16]`	load `w19 = mem[w11 + 16]`
`0x3a0 cmp w20, w10`	compare `j` and array length
`0x3a4 b.lt 0x38c`	if less than, jump to 0x38c

Since the size of an `int` in memory is 4 bytes, index `k` must be multiplied by 4 to find the byte address of `arr[k]` in memory. At instruction 0x394 this is done by the `sxtw #2` which shifts `w19` two places to the left, thereby multiplying it by $2^2 = 4$.

In the example above, there is a rather direct correspondence between the Java program, the bytecode, and the machine code. In general, the correspondence between bytecode and Java code is easy to understand, but the correspondence between machine code and the Java code can be complicated because the Java just-in-compiler attempts to produce highly optimized machine code.

8.2.3 Java run-time stack and heap

A running Java program keeps its data in two distinct parts of memory: the stack and the heap. The stack contains the local variables and parameters of methods, and the heap contains arrays, objects and strings. In particular, all object fields are in the heap.

If a local variable has primitive type such as `int`, `double` or `char`, then its value is stored directly in the stack. If a local variable has reference type (array, object or string), then the stack contains a reference to the actual value, which is in the heap. The stack grows when a method is called and shrinks when it returns. The heap is managed by the garbage collector, which from time to time removes objects that are no longer referred to, so that the memory can be reused for new objects. See [15] for more details.

The Java program below declares three String variables and a `char` variable:

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

```
static void main() {
  String s = "IT University";
  String t = s;
  String itu = s + " of Copenhagen";
  char c = itu.charAt(3);
  ...
}
```

Figure 8.3 shows that when the `main` method has been executed as far as the "`...`" line, the stack will contain a value for each of the four variables, where the value `'u'` for `c` is kept directly in the stack, and the values for the String variables `s`, `t` and `itu` are kept in the heap.

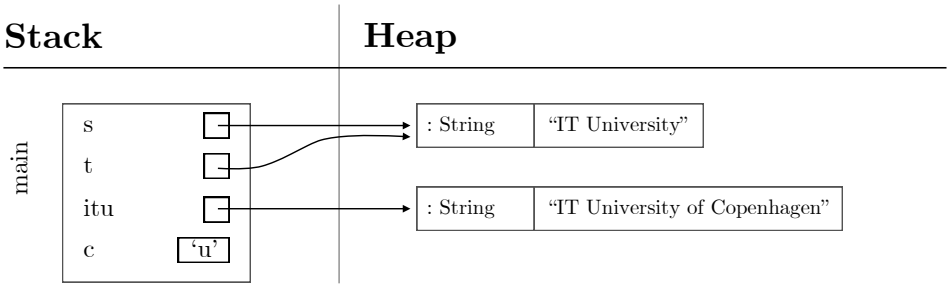


Figure 8.3: Snapshot of a running Java program's memory, divided into stack and heap. The stack contains local variables `s`, `t`, `itu` and `c`, and the heap contains the strings referred to by String-valued variables. Note that two distinct variables may point at the same heap object.

8.3 A single-core CPU with cache memory hierarchy

In reality, a modern CPU is not directly connected to RAM, although this might have been the case before 1990. The reason is that a modern CPU cycle as described above may take as little as 0.25 ns (nanoseconds) whereas modern RAM access takes maybe 100 ns. Hence if the CPU had to execute a `mem[PC]` access as part of each instruction fetch and execution, it would become 400 times slower, which is unacceptable.

8.3.1 Caches, memory and locality

The solution to the huge speed gap between the CPU and RAM is to insert multiple levels of smaller and faster cache memory between the CPU and the RAM, as shown in figure 8.4.

The idea is that accesses to RAM typically exhibit spatial and temporal locality. Spatial locality means that if address 0x000 has just been accessed, then likely address 0x004 will be accessed soon, which is indeed the case in the example machine code program above. Temporal locality means that if address 0x000 has just been accessed, then it will likely be accessed again soon, which is also the case in the example above — in fact, it is the case for any RAM location that contains a program instruction that is part of a loop.

In theory one level of cache between the CPU and RAM would help, if the cache could be made both fast enough and large enough. In practice, to minimize fabrication cost as well as power consumption, two or three levels of cache are used today. The closer to the CPU, the faster and smaller the caches are.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

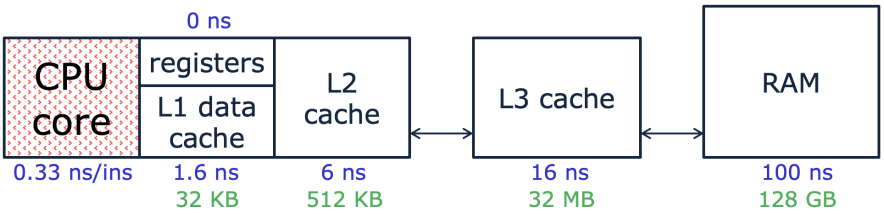


Figure 8.4: Memory hierarchy focusing on a single core of an AMD EPYC 7313 server CPU, showing a core, three levels of cache called L1, L2 and L3, and RAM, and sizes and latencies.

Section 10 of the *Microbenchmarks in Java, C# and C* lecture note shows how to measure the access latency for the various levels of cache in a given machine.

A cache is structured into cache lines, typically of length 64 bytes. This means that when a core needs to access four bytes of data at `arr[0]`, and it is not already in cache, then a cache line of 64 bytes will be transferred from RAM to cache, including the next 15 elements of the array. This is good if those array elements will be used soon, for instance `arr[1]`, but may be wasteful if none of them will be used.

Therefore the layout of data, especially in arrays, can considerably affect program performance by being more or less cache-friendly. In particular, traversing arrays sequentially in address order `arr[0]`, `arr[1]` and so on, is often much faster than traversing them with large strides such as `arr[0]`, `arr[1000]`, `arr[2000]`, and so on.

However, modern CPUs have machinery such as "L1 cache prefetch" that can make smaller strides almost as fast as sequential traversal. Namely, if the prefetch mechanism notices that a program instruction in a loop has accessed equally-spaced memory addresses 0, 1000, 2000 then it guesses that address 3000 may soon be needed also, and speculatively fetches it into cache even before it has been requested by the program instruction. This typically works only up to a certain stride length, maybe 2048 bytes. We can experimentally investigate this effect using our microbenchmarks framework.

Since a cache is much smaller (maybe 1 MB) than RAM (maybe 16384 MB), a given cache line does not correspond to a single memory address but to many possible memory addresses, say 0x0000, 0x2000, 0x4000 and so on. The cache line can only hold data from at most one of these addresses at a time. Therefore a cache line may get evicted — replaced by different data — even though the data it contains is still used by the core. For instance, if the cache line currently holds data from memory address 0x2000 but the core now also needs data from memory address 0x4000, it will evict the cache line's data and replace it with data from address 0x4000. If the core still needs the data from address 0x2000 it must then load it (and the whole cache line) again, thereby evicting the 0x4000 data, and so on, causing poor performance. There are ways make this problem less likely to appear, such as n-way associative caches, which are shown for instance at the bottom and to the right in figure 8.5.

8.3.2 Case: Matrix multiplication and cache performance

We consider algorithms for matrix multiplication [21], widely used in business, computer graphics, machine learning, science, and countless other fields. Here we consider only square N-by-N matrices and represent such a matrix A as a flat array `arr` of N^2 32-bit floats in row-major or-

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

der, such that the matrix element A_{rc} at row r and column c is at array position `arr[r*N+c]` for $0 \leq r < N$ and $0 \leq c < N$.

Given two N-by-N matrices A and B we can compute their product $C = AB$, which is also an N-by-N matrix, like this:

$$C_{rc} = A_{r0} \cdot B_{0c} + A_{r1} \cdot B_{1c} + \cdots + A_{r(N-1)} \cdot B_{(N-1)c}, \quad \text{for} \;\; 0 \leq r < N, 0 \leq c < N$$

It is straightforward to express this computation as a Java method:

```
static void matrixMultRci(float[] A, float[] B, float[] C, int N) {
  for (int r = 0; r < N; r++)
    for (int c = 0; c < N; c++)
      C[r * N + c] = 0.0f;
  for (int r = 0; r < N; r++)
    for (int c = 0; c < N; c++)
      for (int i = 0; i < N; i++)
        C[r * N + c] += A[r * N + i] * B[i * N + c];
}
```

The first set of loops initialize the result matrix C to all zeroes, and the second set of loops accumulate the products $A_{ri} \cdot B_{ic}$ in C_{rc} for all combinations of r and c. Note that the `matrixMultRci` method is named for the order of its inner loop variables: `r`, `c`, `i`. This works and is reasonably fast; when $N = 400$, the matrix multiplication requires $N^3 = 64,000,000$ floating-point multiplications and additions. In theory, a modern 4 GHz CPU core should be able to perform at least $4 \cdot 10^9$ multiplications and additions per second, so the arithmetics alone should be doable in $64 \cdot 10^6 / 4 \cdot 10^9 = 16$ ms or less. In practice, the above method takes around 38 ms, so there must be some overhead from caches and RAM.

Let us focus on the innermost loop, which takes most of the execution time:

```
for (int i = 0; i < N; i++)
  C[r * N + c] += A[r * N + i] * B[i * N + c];
```

When `i` increases by 1, the `C` index stays constant and the `A` index `(r * N + i)` moves to the next element of `A`, which is good for spatial locality. By contrast, the `B` index `(i * N + c)` moves `N` array elements ahead in `B`, which is bad for spatial locality and for cache performance, especially when `N` is large so that the cache prefetch mechanism does not help. However, we can improve on this simply by interchanging the loop order, so it becomes `r`, `i`, `c`:

```
static void matrixMultRic(float[] A, float[] B, float[] C, int N) {
  for (int r = 0; r < N; r++)
    for (int c = 0; c < N; c++)
      C[r * N + c] = 0.0f;
  for (int r = 0; r < N; r++)
    for (int i = 0; i < N; i++)
      for (int c = 0; c < N; c++)
        C[r * N + c] += A[r * N + i] * B[i * N + c];
}
```

Now the inner loop is

```
for (int c = 0; c < N; c++)
  C[r * N + c] += A[r * N + i] * B[i * N + c];
```

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

and we see that when `c` increases by 1, the `C` index moves to the next `C` element, the `A` index stays constant and the `B` index `(i * N + c)` moves to the next `B` element. This improves spatial locality and indeed is much faster, in practice taking only 19.5 ns for $N = 400$. The number of arithmetic operations remains exactly the same, so the difference is caused by better use of the caches in the memory hierarchy. Note that this simple change was possible only because the methods start by initializing `C` to all zeroes and then incrementally builds the product of an `A` row and a `B` column in `C`'s elements, which may seem inefficient but apparently is not.

8.4 A modern CPU core

Each core in a modern CPU pretends to execute instructions one at a time, one cycle per instruction, in the 1980s manner described in section 8.1, but the reality is considerably more complicated and involves internal parallelism and a great deal of electronic machinery.

8.4.1 Internal structure and parallelism of a single core

Figure 8.5 is a diagram of a single modern CPU core from 2019, typically used in smartphones, tablets and laptop computers. The diagram's boxes represent various components (made of transistors) and the arrows represent flow of data between the components (made of wires).

Here are some important concepts and components of a modern CPU core illustrated by Figure 8.5:

- Pipelining: The fetching, decoding and execution of an instruction may be broken into multiple steps (maybe 5 to 15), each executed by a stage of an instruction pipeline. This means that at any given point in time, multiple instructions are overlapping and have been partially executed. The pipeline is indicated by the many arrows that show the flow of instructions and macro- and micro-instructions (MOP and μOP) through the Front End and Execution Engine, from top to bottom in figure 8.5.

- Instruction level parallelism (ILP) or superscalar architecture: To support pipelining, there may be multiple copies of functional units — those that do the actual computing — so that several (usually 2 to 4) integer operations or floating-point operations can be performed per cycle. Some examples are the multiple integer Arithmetic Logic Units (ALU), multiple floating-point FADD, FMUL etc. units, and multiple AGU (Address Generation Units) shown in the lower half of figure 8.5.

- Memory load and store queues: Since memory is so much slower than the CPU, many memory load and store operations may have started and still be ongoing at the same time. This is handled by the Load Buffer and Store Buffer of the LSU (Load-Store Unit) in the lower right corner of the Execution Engine in figure 8.5. The Load and Store Buffers may also try to optimize memory operations by reordering them; this is further discussed in section 8.5.

- Speculative execution: After a conditional jump instruction, such as `b.lt` from the example in section 8.2.2, it is not known at which address the program execution should continue until the preceding instructions (`add` and `cmp`) have finished executing so their

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

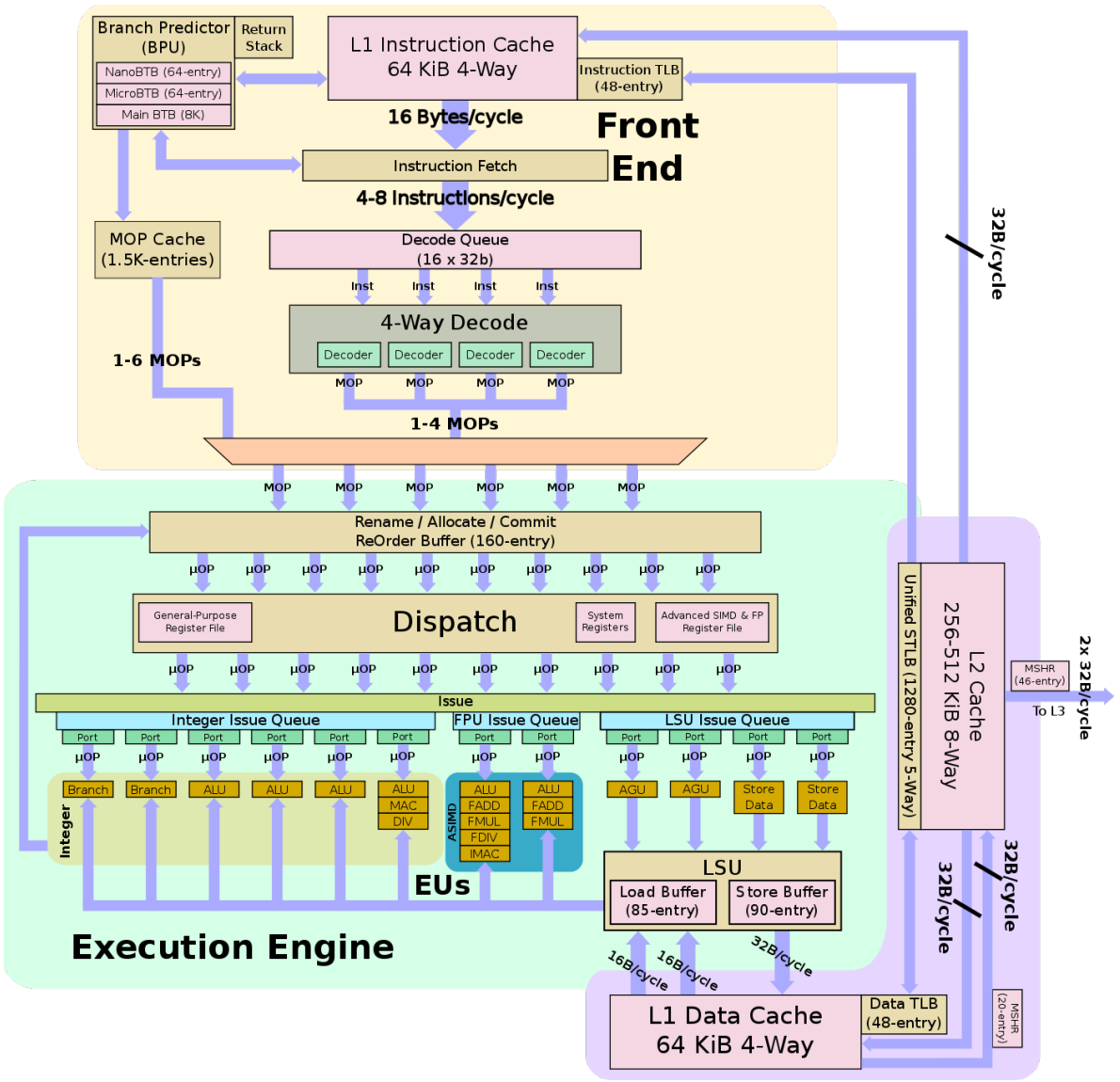


Figure 8.5: The structure of a single Arm Cortex A77 core from 2019. Relevant acronyms: AGU means Address Generation Unit, ALU means Arithmetic-Logic Unit, EUs means Execution Units, FADD means Floating-Point Add (and so on), FPU means Floating-Point Unit, LSU means Load-Store Unit, MOP means macro-operation, μOP means micro-operation. Image author and license unknown.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

values (`w20` and the comparison result) have become known. But if the core always had to wait for this, then the execution of `add`, `cmp` and `b.lt` could not overlap and the pipeline would stall, that is, lack work to do, so the program would execute slowly. Instead it will speculatively execute either the target of the conditional jump or the instruction following the jump. At some later time, the actual outcome of the condition will become known, and then the pipeline can either continue executing the speculatively executed instructions, or it has to discard them and reverse any effects they have had.

The need for such speculation also arises at virtual method calls (which method is actually called?) and by return from methods (which call site should this call return to?).

- Branch prediction: To make sure that speculative execution has a high probability of correctly predicting which way a conditional branch will go, the core contains machinery for branch prediction. This may be as simple as "a branch that goes backwards is likely to be taken" because it will usually be part of a loop, or "if this branch was taken last time it was executed, then it is likely to be taken next time also", or something much more sophisticated. This unit is shown as the Branch Predictor (BPU) in figure 8.5.

- Register renaming: When the execution of multiple instructions that use the same registers overlap, it is necessary to have several copies of each register. This is even more necessary in case of speculative execution, which should be allowed only to update shadow copies of registers until it has been confirmed that the speculative execution was correct. Thus, although the instruction set manual [1] says there exist only registers `x0-x30`, in reality there may be many more in the hardware. This is illustrated by the (Register) Rename aspect of the Reorder Buffer in figure 8.5.

- Hyper-threading (Intel) or simultaneous multithreading (AMD): Instruction execution may stall for a long time when an instruction waits for data from memory (in case of cache misses). Some CPU designs allow a single core to execute two streams of instructions by duplicating only a few components of the core, such as the registers, instruction decoding and the pipeline, while sharing the integer and floating-point functional units and the memory addressing units. When one thread is stalled waiting for memory to respond, then another thread can be run on the other instruction execution machinery. This is not shown in figure 8.5 because that CPU core does not support hyperthreading.

- Vector instructions: Computations on vectors and matrices perform the same numeric operation on many data items and are heavily used in computer graphics, engineering, science, large language models and so on. Vector instructions, also called single instruction multiple data (SIMD), speed up such computations by performing 4 to 16 arithmetic operations per cycle instead of one. Vector instructions are known under brand names such as SSE, AVX, AVX512, and Neon. Hardware vector instructions can be used from Java through the java.incubator.vector package and from C# through the System.Runtime.Intrinsics namespace. They are shown as the ASIMD block in figure 8.5.

It is worth knowing about vector instructions because they can considerably improve performance provided that the given data layout is suitable. It is also worth knowing about branch

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

(mis)prediction because it may explain some surprising performance results, and because conditional branches can sometimes be replaced with so-called conditional moves which may avoid branch misprediction altogether; see section 8.4.2.

Apart from those two cases, it is difficult and rarely worthwhile to try to use knowledge about a core's internal mechanisms to improve program performance. The precise details are often manufacturer secrets and may change from one CPU version to the next. Also, despite all the machinery and parallelism inside the core, the core will try to pretend to execute one instruction at a time in a deterministic manner, which makes it hard to experimentally investigate what goes on. That said, most CPUs have performance monitor counters that can be accessed through programs such as `perf` on Linux or through Java Microbenchmark Harness [9] to better understand a program's behaviours, but we shall not pursue that here.

8.4.2 Case: Branch misprediction

To see how the core's pipeline and branch misprediction can produce surprising performance results, consider bubble sort of an integer array `arr[0..n-1]`, here expressed in the C programming language:

```
void bubblesortcondswap(int* arr, int n) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n-i-1; j++)
      if (arr[j] > arr[j+1]) {
        int tmp = arr[j];  arr[j] = arr[j+1];  arr[j+1] = tmp;
      }
  }
}
```

The code would be exactly the same in Java, except that the method header would be written as `void bubblesortcondswap(int[] arr, int n)`.

Given array size `n`, the `i` loop always performs `n` iterations and the `j` loop performs `n*(n-1)/2` iterations. However, the number of element swaps performed depends on the data in `arr`. If `arr` is already ordered, then the condition `(arr[j] > arr[j+1])` is never true and no swaps are performed; if `arr` is reversed ordered, then the condition is always true and `n*(n-1)/2` swaps are performed, the maximal number possible; and if `arr` contains elements in random order, then the condition is sometimes true and sometimes false, and the number of swaps is somewhere between these two extremes.

Hence with a somewhat naive or 1980s model of a CPU in mind, one would expect that bubble sort is fastest on already-ordered data, slowest on reverse ordered data, and somewhere in between on randomly ordered data. However, on modern CPUs one may find that bubble sorting an array of 10,000 integers may take respectively 13 ms, 13 ms and 25 ms for the three cases. In other words, a reverse sorted array takes precisely as long as an already-sorted one, and random data is much slower, not faster.

The explanation of course is branch prediction in the CPU core: when the condition is never true (already-sorted data) and when it is always true (reverse sorted data), the core's branch prediction component always correctly predicts the conditional branch and therefore the core's execution pipeline never stalls. When the condition is true according to a random pattern (randomly ordered data), the branch prediction is very often wrong, and poor pipeline utilization wastes a lot of cycles on mispredicted branches.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

In some cases, a conditional branch can be replaced by a so-called conditional move, in which case no branch misprediction can happen. We can rewrite bubble sort without the if-statement but with two conditional expressions computing `max` and `min`, like this:

```
void bubblesortcondmove(int* arr, int n) {
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n-i-1; j++) {
      int aj = arr[j], aj1 = arr[j+1],
          max = aj > aj1 ? aj : aj1, min = aj > aj1 ? aj1 : aj;
      arr[j] = min; arr[j+1] = max;
    }
  }
}
```

This does not look like an improvement, but it is: a modern C compiler such as `clang` can compile the expressions for `max` and `min` into conditional move instructions for the CPU hardware, called `csel` in Arm64 and `cmov` in the Intel/AMD x86-64 instruction sets. A conditional move instruction simply chooses between two already-computed values, cannot cause branch misprediction, and ensures good use of the core's pipeline. Indeed, the execution time for an array of 10,000 integers is now 13 ms regardless of the array contents.

At the time of writing (October 2026) the Java just-in-time compiler does not compile the above bubble sort version to conditional moves and therefore this technique does not work in Java, and it also does not work with all C compilers.

No optimization can make bubble sort into a good sorting algorithm, but that is not the point of the example. The point is that some understanding of a modern CPU core can make performance results less mysterious.

8.5 Machine instructions for memory access

The machine code instructions most relevant for performance of programs are those related to memory access: those that load data (or read) from memory, and those that store (or write) data to memory. This is true for both sequential and parallel programs, as illustrated by sections 8.3 and 8.6 and by the case studies in chapter 9.

Here we focus on memory access instructions in Arm64 CPU family, already used in sections 8.1 and 8.2.2, because it better illustrates the various memory effects than the other widely used CPU architecture, the x86-64 family designed by Intel and AMD.

Some important Arm64 memory access instructions are shown in figure 8.6; vastly more details can be found in [1].

The `ldr` instruction loads data from memory to a register, and conversely `str` stores data from a register into memory. In Java terms, `ldr` and `str` are used for reading from and writing to local variables, method parameters, object fields, and array elements.

The `ldar` and `stlr` instructions are variants of `ldr` and `str` which additionally have memory visibility effects on shared-memory multicore machines as described below. In Java terms they are used for reading and writing volatile object fields.

The `ldar` instruction is called "load-acquire" where the "a" stands for *acquire semantics* and the `stlr` is called "store-release" where the "l" stands for *release semantics*. The precise meaning of this is supposed to be given in [12]; here we offer these definitions:

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

Instruction	Meaning	Example Java equivalent
`ldr x10, [x12]`	Load `x10` from `mem[x12]`	`x = arr[k]`
`str x10, [x12]`	Store `x10` to `mem[x12]`	`arr[k] = x`
`ldar x10, [x12]`	As `ldr` but for volatile field `f`	`x = o.f`
`stlr x10, [x12]`	As `str` but for volatile field `f`	`o.f = x`
`ldaddal x10, x11, [x12]`	Atomic add `x10` to `mem[x12]`	`x11 = x12.getAndAdd(x10)`
`casal x10, x11, [x12]`	If `mem[x12]==x10`, set to `x11`	`x12.compareAndSet(x10,x11)`

Figure 8.6: Some Arm64 memory access instructions. The registers `x10` and `x11` could be any 32-bit register `w0-w30` or 64-bit register `x0-x30`, and register `x12` could be any 64-bit register `x0-x30`.

- A load-acquire instruction `ldar` guarantees that all subsequent memory loads and stores start after this load instruction.
- The store-release `stlr` guarantees that all preceding loads and stores complete before this store instruction.

It follows that when a store-release `stlr` is executed before a load-acquire `ldar`, then any load performed after the `ldar` will be able to observe all stores performed before the `stlr` — which is just what Java prescribes when a write to a volatile field is followed by a read from it.

In a naive 1980s computer model where the CPU writes directly to memory, the load-acquire and store-release requirements above trivially hold, and it might seem crazy that they could fail to do so. However, consider a modern CPU core design with a Load Buffer and a Store Buffer as shown in the lower right corner of figure 8.5. These buffers contain load and store operations to be sent through the cache memory hierarchy, so that the core can read from memory and write to memory. The buffers can reorder and bundle `ldr` and `str` operations if this improves memory performance. However, the buffers must always schedule an `ldar` load-acquire instruction *before* any subsequently issued load operations of any kind, and always schedule an `stlr` store-release instruction *after* any previously issued store operations of any kind. This implements acquire and release semantics in a particular core; additional machinery is needed to do this across cores and their caches.

The `ldaddal` instruction implements atomic increment of an integer in a memory location; in Java terms, this has the effect of the `addAndGet` or `getAndAdd` methods from the AtomicInteger and AtomicLong classes. It is indeed used to implement these methods; see section 9.2.4 for an example. The "a" and "l" at the end of `ldaddal` mean that the instruction's reading from memory has acquire semantics and its writing to memory has release semantics. As a consequence, any stores to memory before the `ldaddal` will be visible to any loads after it.

The `casal` instruction implements atomic compare-and-set operations; in Java terms, this has the effect of the `compareAndSet` methods from the java.util.concurrent.atomic classes. It is indeed used to implement these methods; see section 9.2.5 for an example. The "a" and "l" in `casal` mean the same as in `ldaddall` above, and have the same visibility effects.

The load-acquire effect of `ldar` is called a half-barrier or half-fence, because it only restricts *subsequent* memory loads and stores to start after the `ldar`, whereas loads and stores that *precede* it may still be moved after it if the CPU core prefers to. Similarly, `stlr` permits subsequent reads and writes to move before it. A full memory barrier or full fence is an

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

instruction that prevents any movement of memory loads and stores across the barrier, which may harm performance more than the half-barriers. These details are of no concern to Java programmers, where the memory visibility is defined in terms of volatile fields, locks, atomic operations and so on, but when programming in C#, C or C++ one needs to be aware of these complications.

8.6 Multiple cores and caches

A modern CPU has multiple cores of the sort described in section 8.4.1, typically between 2 and 128. Since each core can execute program code independently of the others, the CPU can run multiple programs truly in parallel; this is called multiple instruction multiple data (MIMD) parallelism. This is in contrast to a 1980s CPU that could typically only simulate parallel program execution by interleaving the execution of instructions from multiple programs.

Let us consider the structure of CPU, caches and RAM for two concrete CPUs:

- A Macbook Pro laptop computer with an Apple M3 pro CPU (launched 2023) with Arm64 instruction set. Its structure is shown in figure 8.7.
- A server computer with an AMD EPYC 7313 CPU (launched 2021) with x86-64 instruction set. Its structure is shown in figure 8.8. Note that a given core, for instance L#1, has its own L1 and L2 caches, shares an L3 cache with three other cores, and is part of a chip (package) that contains three other blocks with four cores each. Presumably two cores in the same block, say L#1 and L#3, can coordinate their cache contents faster than two cores in different blocks, say L#1 and L#7, and much faster than two cores on different chips, say L#1 and L#23, which must send data out of the chip to do so.

It is worth remembering that each of the many cores in these two figures has internal complexity like that shown in figure 8.5 and described in section 8.4.1. Modern computer hardware is extremely complex: the laptop M3 pro CPU chip has 37 billion transistors, which is 134,000 times more than the first CPU that could run the Linux operating system back in 1991. It is nearly impossible to understand all the details of such a system.

These two machines mentioned here are used in the scalability experiments in chapter 9.

8.7 Cache coherence and the MESI protocol

Each of the multiple cores shown in figures 8.7 and 8.8 is connected to the same RAM, through layers of core-specific or shared caches. This creates a new challenge: how to keep track of where the "authoritative" version of a memory location's contents — typically corresponding to an object field or array element in Java — is stored? Is it in the RAM itself, or in a cache, and if so which cache? How do we avoid that two caches hold conflicting values for a given memory location? All this is taken care of by a *cache coherence protocol*, which exchanges messages and data between the cores, caches and RAM.

Here we describe the MESI cache coherence protocol for multicore machines [14]. As mentioned in section 8.3.1, a cache is divided into cache lines. In the MESI protocol each cache line has one of four possible states called M, E, S and I, giving the protocol its name:

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

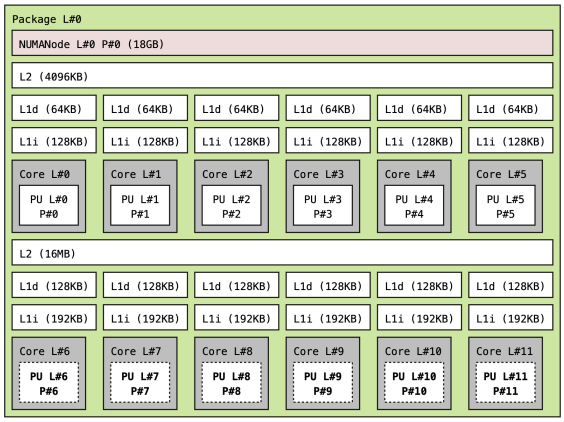


Figure 8.7: Structure of a laptop CPU, Arm64 Apple M3 pro. There are 6 "efficiency" cores L#0 to L#5 and 6 "performance" cores L#6 to L#11. Each performance core has a 128 KB L1 data cache and a 192 KB L1 instruction cache, and all performance cores share a 16 MB L2 cache. The efficiency cores have less L1 and L2 cache. No L3 cache is shown, but there is a system-level cache shared between CPU and on-chip GPU. Image by `lstopo` [16].

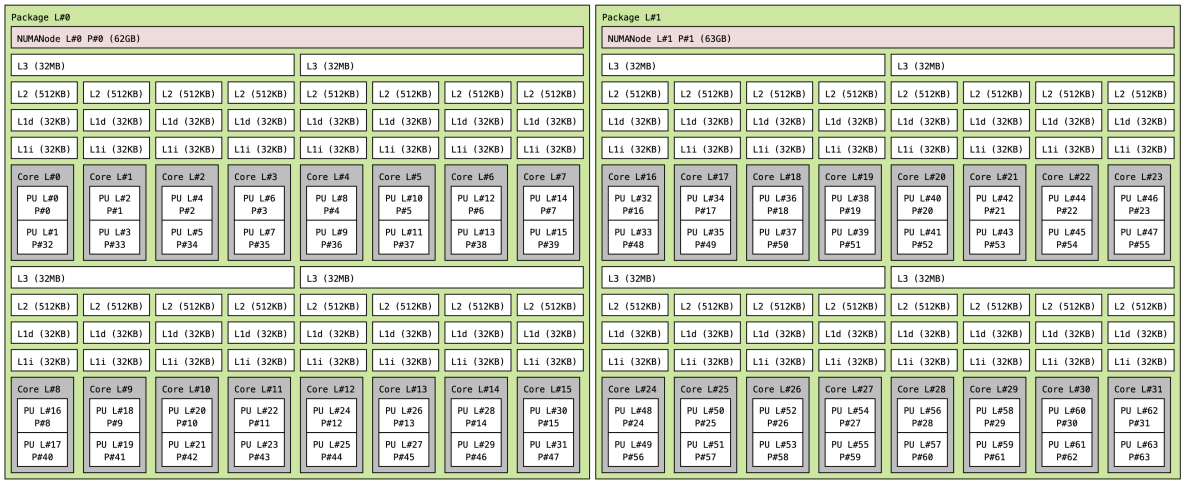


Figure 8.8: Structure of a server CPU, AMD EPYC 7313. There are 2 chips (packages), each with 4 blocks each with 4 cores, in total $2 \cdot 4 \cdot 4 = 32$ cores named L#0 to L#31. Each core has a 32 KB L1 data cache, a 32 KB L1 instruction cache and a 512 KB L2 cache. Each block of 4 cores share a 32 MB L3 cache. Each core supports simultaneous multithreading and therefore has two processing units (PU). Image by `lstopo` [16].

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

M Modified: This cache line holds modified data, the corresponding memory address is invalid in all other caches and in RAM, the core can update the data, and the core must write it back to RAM when evicting the cache line.

E Exclusive: This cache line holds unmodified data from RAM, the corresponding memory address is invalid in all other caches but valid in RAM, and there is no need to write the cache line back to RAM when evicted.

S Shared: This cache line holds unmodified data from RAM, other caches may have the same memory addresses' data in S state, and there is no need to write the cache line back to RAM when evicted.

I Invalid: The data in this cache line does not correspond to any memory location.

A core that needs to compute `(count + 1)` where `count` is not in cache, must fetch `count` into a cache line which is put in the S state. If next the core needs to update `count` with the result of `(count + 1)`, then it must first change the cache line status to M so that it can modify it (and at some later point write it back to RAM). These cache line state changes require communication with the other caches and RAM, via a memory bus. For instance, when a cache line moves from the S state to the M state, all other caches must be told to invalidate that cache line, if they have it.

Figure 8.9 summarizes the four states and the communication actions needed to read or write data on a cache line in that state.

State	Cache line	RAM	To read	To write	When evicted
Modified	Modified by me	stale	from cache	to cache	write to RAM
Exclusive	Not modified	fresh	from cache	to cache -> M	-
Shared	Others have it too	fresh	from cache	send invalidate	-
Invalid	Not in use by me	-	elsewhere	send invalidate	-

Figure 8.9: For each cache line state we show its meaning, whether RAM also holds the up-to-date value, what to do to read from the location, what to do to write to it, what to do when the cache line is evicted (replaced). Here "me" is the core that the cache belongs to.

There are 12 possible transitions between the 4 cache line states, labelled a to l (ell) in figure 8.10. Figure 8.11 describes the 12 transitions, specifies when each must be done, and describes what communication with other caches and RAM the transition requires.

Here are some notable consequences of the need for cache coherence:

- When multiple cores want to read the same memory location but not modify it, they can all keep it in cache in the S state, without exhanging any messages, which is efficient. Hence shared immutable data — as in functional programming — can be efficient.

- When a single core wants to read and write the same memory location repeatedly, but no other core reads or writes it at the same time, that core can keep it in cache in state M without exchanging any messages, which is efficient. Hence unshared, or core-local, mutable data — as in message-passing concurrency — can be efficient.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

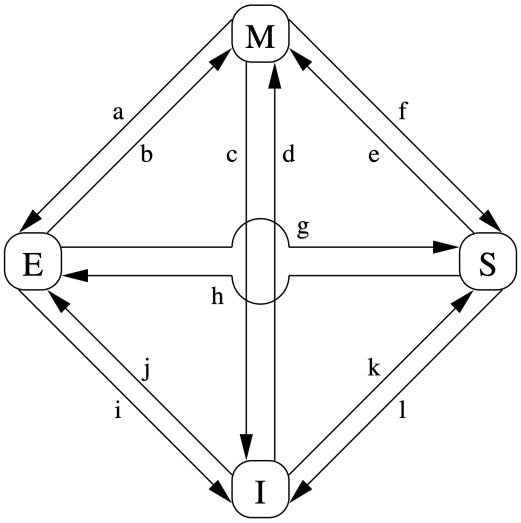


Figure 8.10: The 12 possible transitions a–l between the 4 MESI cache line states. From [14].

Fr	To	Lb	Reason for transition	I send	I receive	My response
M	E	a	(Send update to RAM)	writeback	-	-
E	M	b	I want to write	-	-	-
M	I	c	Other wants to write	-	read inv	read resp, inv ack
I	M	d	Atomic read-mod-write	read inv	read resp, inv ack*	-
S	M	e	Atomic read-mod-write	read inv	inv ack*	-
M	S	f	Other wants to read	-	read	read resp
E	S	g	Other wants to read	-	read	read resp
S	E	h	I will soon write	inv	inv ack*	-
E	I	i	Other wants atomic rw	-	read inv	read resp, inv ack
I	E	j	I want to write	read inv	read resp, inv ack*	-
I	S	k	I want to read	read	read resp	-
S	I	l	Other wants to write	-	inv	inv ack

Figure 8.11: The 12 possible transitions for a cache line's MESI state. For transitions (abdehjk) initiated by the core, the table shows the message sent by the core and the responses it expects to receive from other cores. For transitions (cfgil) initiated by another core, the table shows the message received and the responses that this core sends. Here, "ack" means acknowledge, "ack*" means multiple acknowledge messages, "inv" means invalidate, "resp" means response. As before, "me" is the core that the cache belongs to. Based on [14].

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15

- When multiple cores want to read and write the same memory location repeatedly, each of them must get the location in cache in state M, thereby both repeatedly invalidating all other cores' copies of that location and causing lots of read-invalidate and invalidate-acknowledge messages, which is inefficient. Hence mutable data shared between multiple cores can be very inefficient.

There are a couple of subtler points worth mentioning.

First, a software thread runs on a hardware core. A good operating system or virtual machine tries to make sure that a thread has affinity to a particular core, so that whenever the thread runs, it runs on the same core. This increases the chance that the core's cache contains some data relevant to the thread from last time the thread executed on that core.

If, thanks to such affinity, a given thread mostly runs on the same core, then the above observation about core-local mutable data being efficient implies that thread-local mutable data should be efficient too.

Second, if many threads want to read and write the same data repeatedly, that will likely cause the executing cores to repeatedly invalidate each others' cache lines, send many memory bus messages, and therefore be inefficient.

Hence mutable data should be mostly thread-local when possible. Local variables in methods are always thread-local, but object fields and array elements are sharable between threads.

Immutable data can be shared between threads without loss of performance.

Third, different computers have different cache coherence machinery and overhead. In general, a small CPU with few cores may have simpler and faster cache coherence than a large CPU with many cores. Therefore cache coherence overhead may be a worse problem on a large CPU, so that a concurrent data structure that scales well on a laptop may scale very badly on a large expensive server CPU.

Comparing the laptop and server CPUs shown earlier, it is clear that the laptop CPU in figure 8.7 has fewer cores but is highly integrated on a single chip, whereas the server CPU in figure 8.8 has more cores but presumably longer communication paths between cores in different blocks or on different chips. Hence one might expect the laptop to scale very well up to 6 threads (running on the 6 performance cores) even when there is much cache coherence traffic, whereas the server might scale well up to 32 threads (running on 32 cores) only when the mutable data is mostly thread-local and therefore core-local and (L1, L2) cache-local.

Fourth, as mentioned before, a cache line typically contains 64 bytes of data, corresponding to 16 integers, so multiple object fields may be stored on the same cache line. Let us say that we have two object fields, `mean` and `count`, where `mean` is immutable and `count` is mutable, on the same cache line, and two cores C0 and C1 each has a copy of this cache line in the S state. In the MESI protocol, a cache line is treated as a whole. If core C0 wants to update the `count` field and so needs the cache line in state M, it tells core C1 to invalidate the cache line whereby C1 loses the the value of field `mean`. This phenomenon is called false sharing. Fields `mean` and `count` may not have anything to do with each other, but by accident they are on the same cache line, and field `mean` becomes an innocent victim of core C0's wish to update fields `count`. This happens in practice and can lead to some performance problems that are hard to diagnose, as we shall see in section 9.4.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-15